

HayPaComer

Nevera inteligente para reducir desperdicio y decidir qué cocinar

Propuesta de proyecto en Java, IoT e inteligencia artificial

Septiembre de 2026

Resumen

HayPaComer es un sistema doméstico que combina una aplicación Java, un módulo ESP32 y una pesa de cocina para mantener un inventario vivo de la nevera. Su función principal no es mostrar recetas genéricas, sino responder una pregunta cotidiana: “¿qué puedo cocinar ahora con lo que realmente existe en casa?”. El sistema considera cantidades medidas en gramos, vencimientos, preferencias, propiedad de los alimentos, tiempo disponible y eventos de temperatura. La inteligencia artificial reduce la fricción de registrar productos y redactar sustituciones, pero las cantidades, conversiones y reglas de seguridad permanecen disponibles sin conexión.

Índice

1. Problema y oportunidad	2
2. Propuesta de valor y funcionalidades	2
3. Arquitectura propuesta	2
4. Hardware y protocolo	3
5. Módulo de cantidades y sustituciones	4
6. Inteligencia artificial	4
7. Patrones de diseño justificados	5
8. Modelo Java inicial	5
9. Plan de implementación	6
10. Demo y criterios de éxito	6
11. Precedentes y diferenciación	6

1. Problema y oportunidad

En una vivienda se compra comida que luego queda escondida, se duplican productos y se olvidan sobras. La aplicación típica de recetas parte de una despensa perfectamente registrada; una casa real tiene paquetes abiertos, recipientes sin etiqueta y alimentos de distintos integrantes. Además, una puerta abierta durante mucho tiempo o un apagón puede comprometer productos perecederos.

El problema se formula así: **la familia no tiene una representación confiable, actualizada y accionable de su comida**. HayPaComer transforma esa información en decisiones pequeñas: consumir primero una sobra, preparar un plato de 15 minutos o añadir un producto a la lista de mercado.

Objetivo general. Diseñar e implementar en Java un asistente doméstico que registre el estado de una nevera mediante interacción manual y sensores, recomiende comidas factibles y ayude a disminuir desperdicio y compras repetidas.

Usuarios. Familias, compañeros de vivienda y estudiantes que comparten nevera. El sistema se diseña para una casa, varios usuarios y una nevera física.

2. Propuesta de valor y funcionalidades

1. **Inventario vivo:** registrar producto, cantidad, unidad, zona, fecha de vencimiento y propietario.
2. **Cocina ahora:** indicar minutos disponibles, número de personas, equipo disponible y restricciones; recibir hasta tres opciones que se puedan hacer realmente.
3. **Modo rescate:** priorizar sobras y productos próximos a vencer antes de sugerir compras.
4. **Lista de mercado colaborativa:** descontar ingredientes al cocinar y agregar automáticamente faltantes agrupados por producto.
5. **Sensado de puerta:** detectar apertura, duración y frecuencia; alertar después de un umbral configurable.
6. **Cadena de frío:** registrar temperatura y periodo fuera de rango. El sistema marca un alimento como *en revisión*, no inventa una garantía sanitaria.
7. **Módulo de pesa de dos modos:** en modo Nevera actualiza el stock real de un producto; en modo Cocina pesa el ingrediente que se va a usar.
8. **Cálculo por gramos:** comparar la cantidad medida con la requerida por la receta, escalar porciones y calcular faltantes.
9. **Sustituciones verificables:** si falta un ingrediente, proponer un equivalente y pedir que también se pese antes de aceptarlo.
10. **Propiedad y permisos:** no sugerir el queso de otra persona si el usuario no tiene permiso.
11. **Aprendizaje doméstico:** aprender platos aceptados, horarios y cantidades habituales sin modificar silenciosamente el inventario.
12. **Funcionamiento degradado:** si se cae WiFi o el servicio de IA, continúan inventario local, alertas y recomendaciones por reglas.

3. Arquitectura propuesta

La solución se divide en cuatro partes:

- **Dispositivo:** ESP32 lee reed switch, DS18B20, HX711 y controla LED/buzzer. Envía eventos JSON por WiFi. La misma celda de carga tiene modo Nevera y modo Cocina.
- **Backend Java:** recibe eventos, valida lecturas, mantiene reglas de inventario y expone la fachada del sistema. Puede implementarse con Spring Boot y una API REST sencilla.
- **Persistencia:** SQLite para prototipo; tablas para alimentos, eventos de sensores, recetas, usuarios y lista de mercado.
- **Interfaz:** JavaFX para demo de escritorio o una interfaz web servida por el backend. La pantalla principal muestra “Ahora”, “Nevera” y “Mercado”.

```
ESP32 -> SensorGateway -> NeveraFacade -> InventoryService
                                           -> SafetyService
                                           -> ScaleService
                                           -> MealRecommendationService
                                           -> ShoppingListService
                                           -> SQLite
```

El ESP32 no decide si un alimento es seguro, no calcula sustituciones y no genera recetas: mide gramos, estabilidad, puerta y temperatura. Java normaliza eventos, guarda lecturas y aplica las políticas. Esta separación permite sustituir hardware real por un simulador durante las pruebas.

4. Hardware y protocolo

El ESP32 se instala fuera de la zona fría para evitar condensación; dentro solo entran la sonda y los cables protegidos. Componentes del prototipo:

Componente	Responsabilidad
ESP32	Conectividad WiFi y lectura de sensores.
Reed switch + imán	Estado de la puerta.
DS18B20	Temperatura interna con sonda protegida.
Celda de carga + HX711	Pesa una bandeja en modo Nevera o una porción en modo Cocina.
LED RGB y buzzer	Señal local: correcto, advertencia o alarma.
Fuente 5 V externa	Alimentación estable; no se usa una batería suelta dentro.

Ejemplo de evento:

```
{"device":"fridge-01","mode":"COOK","door":"CLOSED",
  "tempC":5.2,"grams":132,"stable":true,
  "ingredient":"onion","at":"2026-09-04T18:40:00Z"}
```

El backend valida rango, fecha, identificador, modo, estabilidad y duplicados. Antes de pesar se realiza tara para descontar el peso del recipiente. Una lectura solo se acepta cuando permanece estable durante aproximadamente un segundo. En modo Nevera, el cambio de peso actualiza el producto asignado; en modo Cocina, la lectura se compara con la cantidad requerida por la receta. Si no existe un producto asignado o la lectura es inestable, Java solicita confirmación y no modifica silenciosamente el inventario.

5. Módulo de cantidades y sustituciones

La pesa convierte el inventario de una aproximación manual en una cantidad comprobable. La aplicación conserva dos estados relacionados pero distintos:

- **Modo Nevera:** la celda permanece bajo una bandeja configurada para un producto o recipiente. Si el peso pasa de 842 g a 650 g, Java registra un consumo de 192 g, descontando el peso del recipiente conocido.
- **Modo Cocina:** la bandeja se retira o se usa en el mesón. La aplicación solicita un ingrediente, tara el recipiente, espera una lectura estable y compara los gramos pesados contra la receta.

Cada ingrediente posee una cantidad, unidad, peso del envase, porciones y reglas de sustitución. Una receta no se considera disponible solo porque “hay cebolla”: debe existir suficiente cantidad medida o una alternativa aceptada. Si una receta pide 200 g de pollo y solo se pesan 80 g, el motor puede reducir las porciones o buscar una sustitución registrada, por ejemplo atún. El usuario pesa el atún y el sistema vuelve a verificar si la combinación alcanza.

El flujo de cocina es: seleccionar receta, mostrar cantidades requeridas, pesar ingrediente, aceptar o repetir, calcular faltantes, confirmar sustituciones y descontar únicamente lo utilizado. Las sustituciones se modelan con una proporción, límites y contexto; no se permite que una respuesta de IA invente equivalencias peligrosas.

```
ScaleService -> tare() -> stableReading() -> MeasuredQuantity
MeasuredQuantity + RecipeRequirement -> QuantityEvaluator
QuantityEvaluator -> Enough | ScaleDown | Substitute | Missing
```

6. Inteligencia artificial

La IA es un módulo reemplazable, no una dependencia crítica. Se define la interfaz `RecommendationEngine` con dos implementaciones:

- **RuleBasedRecommendationEngine:** funciona offline usando coincidencia de ingredientes, tiempo y restricciones.
- **GeminiRecommendationAdapter:** llama a un modelo multimodal o de texto mediante HTTP, recibe JSON y valida la respuesta.

Casos de uso:

1. **Sugerencia de plato:** Java envía inventario permitido, minutos, personas, cantidades medidas y equipo. La IA devuelve nombre, pasos, ingredientes usados y faltantes en un esquema JSON estricto.
2. **Registro rápido:** OCR de una etiqueta o recibo propone nombre, cantidad y fecha; el usuario confirma antes de guardar.
3. **Sustitución explicada:** la IA puede sugerir que el atún reemplace al pollo según el catálogo permitido, pero Java valida la proporción, el peso medido, restricciones y alergias.
4. **Explicación del estado:** redacta “consume hoy” o “revisa este producto”, pero el nivel de riesgo lo determina una regla explícita de temperatura y tiempo.

La IA no diagnostica alimentos, no decide por sí sola que algo es seguro, no inventa gramos y no recibe datos personales innecesarios. La respuesta debe devolver referencias a ingredientes existentes, cantidades máximas y una justificación. Las claves de API se guardan en variables de entorno y nunca en el repositorio.

7. Patrones de diseño justificados

Patrón	Aplicación real
Facade	HayPaComerFacade ofrece queCocinoAhora(), registrarConsumo() y estadoDeLaNevera() sin exponer subsistemas.
Adapter	Convierte JSON del ESP32, OCR e IA externa al modelo Java común.
Bridge	Separa fuente de medición (puerta, peso, temperatura) de interpretación (puerta abierta, stock, cadena de frío y cantidad utilizable).
Proxy	Cachea el hardware cuando está offline y controla alimentos privados.
Decorator	Añade estados como vencido, sobra, en riesgo o propiedad de un usuario.
Composite	Representa nevera, zona, bandeja y alimento como una estructura navegable.
Builder	Construye una sugerencia con tiempo, personas, restricciones, cantidades medidas y sustituciones.
Strategy	Cambia el criterio de evaluación: por gramos, por unidades, reducción de porciones o sustitución permitida.
Factory Method	Crea eventos de puerta, peso o temperatura y tipos de alimento.
Flyweight	Comparte metadatos de alimentos repetidos; cantidad y fecha permanecen por producto.
Singleton	Una sesión coordinada para la nevera física, evitando dos estados activos.

8. Modelo Java inicial

```
interface SensorAdapter { SensorEvent read(); }
class Esp32SensorAdapter implements SensorAdapter
class SimulatedSensorAdapter implements SensorAdapter
class FridgeFacade
class InventoryService
class ColdChainService
class ScaleService
class MeasuredQuantity
class QuantityEvaluator
class RecipeRequirement
class SubstitutionRule
class IngredientSubstitution
interface RecommendationEngine
class RuleBasedRecommendationEngine implements RecommendationEngine
class GeminiRecommendationAdapter implements RecommendationEngine
class FoodItem; class Recipe; class MealSuggestion
class ShoppingList; class HouseholdMember
```

Una recomendación debe incluir evidencia: alimentos usados, gramos requeridos, gramos medidos, faltantes, sustituciones, tiempo estimado y motivo de prioridad. Así el usuario puede corregirla y el sistema puede probarse sin depender de una respuesta creativa.

9. Plan de implementación

1. Semana 1: modelo de dominio, casos de uso y persistencia SQLite.
2. Semana 2: fachada, inventario, recetas y lista de mercado con pruebas unitarias.
3. Semana 3: ESP32 con reed y DS18B20; simulador de eventos para pruebas.
4. Semana 4: HX711, calibración, tara, lectura estable y recepción de eventos REST.
5. Semana 5: modo Nevera, modo Cocina, evaluación en gramos y reglas de sustitución.
6. Semana 6: motor de reglas, Adapter de IA con respuesta JSON validada y JavaFX.
7. Semana 7: manejo offline, integración, pruebas de ruido y demo.

10. Demo y criterios de éxito

Se carga una nevera con huevos, arroz, sopa de ayer, leche y queso de Ana. Se abre la puerta durante 40 segundos: suena la alarma. En modo Nevera se retira la leche de la bandeja: Java calcula el consumo real descontando el recipiente. En modo Cocina se elige arroz con pollo para dos personas: la receta pide 200 g de pollo, pero se pesan 80 g. El sistema propone reducir a una porción o pesar atún como sustituto. Si se pesan 130 g de atún, valida la alternativa, descuenta los gramos utilizados y genera el faltante. Se simula una temperatura elevada: los perecederos aparecen en revisión. Finalmente, al pedir “15 minutos, Juan, una persona”, la aplicación recomienda sopa o huevos, no el queso ajeno ni el alimento en riesgo.

Indicadores: inventario actualizado, evento de sensor procesado, receta que respeta restricciones, caída de IA sin caída del sistema y lista sin duplicados.

11. Precedentes y diferenciación

Samsung Family Hub y proyectos como DecayDock demuestran la viabilidad de visión e inventario inteligente. FridgeVision y SuperCook demuestran el interés por recomendar recetas a partir de ingredientes. Trabajos recientes con ESP32-CAM también combinan detección, temperatura y visualización.

HayPaComer no pretende competir con una nevera comercial: su diferencia es el contexto doméstico latinoamericano, la bandeja barata, la propiedad de alimentos, el modo apagón y la recomendación concreta de 5/15/30 minutos con fallback offline. Es un prototipo verificable, no una promesa de seguridad alimentaria.

Referencias

- [1] Nong et al., “A smart fridge with AI-enabled food computing”, arXiv:2509.07400, 2025. <https://arxiv.org/abs/2509.07400>
- [2] Samsung, “Family Hub / AI Vision Inside”. <https://www.samsung.com/us/explore/family-hub-refrigerator/overview/>
- [3] FridgeVision, “Turn ingredients into recipes”. <https://fridgevision.app/>
- [4] Hackster, “DecayDock: The Tiny AI Device That Combats Food Waste”, 2026. <https://www.hackster.io/news/decaydock-the-tiny-ai-device-that-combats-food-waste-3a34ddad9cbf>